

Java Programming Notes

Core Concepts, Syntax & Reference Guide

Covers: Basics · OOP · Data Types · Control Flow · Arrays & Strings · Collections · Exception Handling ·
Multithreading · File I/O

1. Introduction to Java

Java is a class-based, object-oriented, platform-independent programming language developed by Sun Microsystems (now owned by Oracle). Java code compiles to **bytecode**, which runs on the Java Virtual Machine (JVM) — this is what makes Java "write once, run anywhere."

Key Features

- **Platform Independent** — bytecode runs on any device with a JVM
- **Object-Oriented** — everything (except primitives) is treated as an object
- **Strongly Typed** — every variable and expression has a defined type
- **Automatic Memory Management** — garbage collector handles memory cleanup
- **Multithreaded** — built-in support for concurrent programming
- **Secure** — no explicit pointers; runs inside a sandboxed JVM

Basic Program Structure

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

Every Java application needs at least one class with a `main` method — this is the entry point of the program.

2. Variables & Data Types

Primitive Data Types

Type	Size	Default	Example
byte	8-bit	0	byte b = 100;
short	16-bit	0	short s = 20000;
int	32-bit	0	int i = 100000;
long	64-bit	0L	long l = 100000L;
float	32-bit	0.0f	float f = 3.14f;
double	64-bit	0.0d	double d = 3.14159;
char	16-bit	'\u0000'	char c = 'A';
boolean	1-bit	false	boolean flag = true;

Reference Types

Reference types store the memory address of an object rather than the value itself. Examples include String, arrays, and any class or interface type.

Type Casting

```
// Widening (implicit) - smaller to larger type
int i = 100;
long l = i;
```

```
// Narrowing (explicit) - larger to smaller type
double d = 100.04;
int j = (int) d;    // j = 100
```

3. Operators & Control Flow

Operator Categories

- **Arithmetic:** + - * / %
- **Relational:** == != > < >= <=
- **Logical:** && || !
- **Assignment:** = += -= *= /=
- **Bitwise:** & | ^ ~ << >>
- **Ternary:** condition ? valueIfTrue : valueIfFalse

Conditional Statements

```
if (score >= 90) {
    grade = "A";
} else if (score >= 75) {
    grade = "B";
} else {
    grade = "C";
}
```

```
// Switch statement
switch (day) {
    case 1 -> System.out.println("Monday");
    case 2 -> System.out.println("Tuesday");
    default -> System.out.println("Other day");
}
```

Loops

```
// for loop
for (int i = 0; i < 5; i++) {
    System.out.println(i);
}
```

```
// while loop
int i = 0;
while (i < 5) {
```

```

        System.out.println(i);
        i++;
    }

    // do-while loop (executes at least once)
    do {
        System.out.println(i);
        i++;
    } while (i < 10);

    // enhanced for-loop (for-each)
    for (int num : numbers) {
        System.out.println(num);
    }

```

4. Object-Oriented Programming

The Four Pillars

- **Encapsulation** — bundling data and methods together, restricting direct access via private fields and public getters/setters
- **Inheritance** — a class acquiring properties and behavior of another class using extends
- **Polymorphism** — same method behaving differently (method overloading & overriding)
- **Abstraction** — hiding implementation details using abstract classes and interfaces

Class & Object Example

```

public class Car {
    private String model;
    private int speed;

    // Constructor
    public Car(String model, int speed) {
        this.model = model;
        this.speed = speed;
    }

    public void accelerate() {
        speed += 10;
    }

    public String getModel() {
        return model;
    }
}

// Creating an object
Car myCar = new Car("Tesla", 0);
myCar.accelerate();

```

Inheritance

```

class Animal {
    void eat() { System.out.println("This animal eats food"); }
}

class Dog extends Animal {
    void bark() { System.out.println("The dog barks"); }
}

Dog d = new Dog();
d.eat();    // inherited method
d.bark();  // own method

```

Method Overloading vs Overriding

Overloading	Overriding
Same class, same method name, different parameters	Subclass redefines a parent class method
Resolved at compile time	Resolved at runtime
No inheritance required	Requires inheritance

Interfaces & Abstract Classes

```

interface Shape {
    double area();           // abstract method
}

abstract class Vehicle {
    abstract void move();
    void fuelUp() {           // concrete method allowed
        System.out.println("Fueling...");
    }
}

class Circle implements Shape {
    double radius;
    Circle(double r) { radius = r; }
    public double area() { return Math.PI * radius * radius; }
}

```

5. Arrays & Strings

Arrays

```

int[] numbers = {1, 2, 3, 4, 5};
int[] scores = new int[10];    // default values = 0

// 2D array
int[][] matrix = new int[3][3];
matrix[0][0] = 1;

// Array length

```

```
System.out.println(numbers.length);
```

Common String Methods

Method	Description
length()	Returns the string length
charAt(i)	Character at index i
substring(a, b)	Substring from index a to b
equals(str)	Compares content of two strings
toUpperCase()	Converts to uppercase
split(regex)	Splits string into an array
trim()	Removes leading/trailing whitespace
contains(str)	Checks if substring exists

Note: Strings in Java are **immutable** — every modification creates a new String object. Use `StringBuilder` for frequent modifications:

```
StringBuilder sb = new StringBuilder();
sb.append("Hello").append(" ").append("World");
System.out.println(sb.toString());
```

6. Collections Framework

Core Interfaces

Interface	Implementations	Notes
List	ArrayList, LinkedList	Ordered, allows duplicates
Set	HashSet, TreeSet, LinkedHashSet	No duplicates
Map	HashMap, TreeMap, LinkedHashMap	Key-value pairs
Queue	LinkedList, PriorityQueue	FIFO order

Usage Examples

```
List<String> names = new ArrayList<>();
names.add("Alice");
names.add("Bob");
```

```
Map<String, Integer> ages = new HashMap<>();
ages.put("Alice", 30);
int age = ages.get("Alice");
```

```
Set<Integer> uniqueNumbers = new HashSet<>();
```

```

uniqueNumbers.add(10);
uniqueNumbers.add(10);    // ignored - duplicate

// Iterating
for (String name : names) {
    System.out.println(name);
}

```

7. Exception Handling

Exceptions represent abnormal conditions during program execution. Java uses try-catch-finally blocks to handle them gracefully.

```

try {
    int result = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println("Error: " + e.getMessage());
} finally {
    System.out.println("This always executes");
}

// Custom exception
class InvalidAgeException extends Exception {
    public InvalidAgeException(String message) {
        super(message);
    }
}

// Throwing an exception
if (age < 0) {
    throw new InvalidAgeException("Age cannot be negative");
}

```

Checked vs Unchecked Exceptions

- **Checked** — must be handled or declared (e.g. IOException, SQLException)
- **Unchecked** — occur at runtime, not enforced by compiler (e.g. NullPointerException, ArithmeticException)

8. Multithreading

Java supports concurrent execution of two or more threads for maximum CPU utilization.

```

// Extending Thread
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread running");
    }
}

MyThread t = new MyThread();
t.start();

// Implementing Runnable (preferred)

```

```

Runnable task = () -> System.out.println("Running via lambda");
Thread t2 = new Thread(task);
t2.start();

// Synchronization to avoid race conditions
synchronized void updateBalance() {
    balance += 100;
}

```

9. File I/O

```

import java.io.*;

// Writing to a file
try (FileWriter writer = new FileWriter("output.txt")) {
    writer.write("Hello, File!");
} catch (IOException e) {
    e.printStackTrace();
}

// Reading from a file
try (BufferedReader reader = new BufferedReader(new FileReader("output.txt"))) {
    String line;
    while ((line = reader.readLine()) != null) {
        System.out.println(line);
    }
} catch (IOException e) {
    e.printStackTrace();
}

```

Using try-with-resources (as shown above) automatically closes the resource, even if an exception occurs.

10. Quick Reference — Access Modifiers

Modifier	Class	Package	Subclass	World
public	Y	Y	Y	Y
protected	Y	Y	Y	N
default	Y	Y	N	N
private	Y	N	N	N

Keywords Cheat-Sheet

- **static** — belongs to the class rather than any instance
- **final** — constant value / non-overridable method / non-extendable class
- **this** — refers to the current object instance

- **super** — refers to the parent class
- **instanceof** — checks object type at runtime